Dockerized Entertainment React Service

Course Name: Devops

Institution Name: Medicaps University - Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	PRABAL SHUKLA	EN22CS301713
2.	PRATIK KOUSHAL	EN22CS301747
3.	PRIYANSH CHOUHAN	EN22CS301759
4.	PUSHPRAJ SINGH CHOUHAN	EN22CS301774
5.	RAJAT MALVIYA	EN22CS301783

Group Name:11D7

Project Number:DO-34

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year: 2025-2026

Table of Contents

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

Modern web applications often face challenges related to deployment, environment configuration, scalability, and maintenance. Traditional application deployment methods require manual installation of dependencies and environment setup, which often leads to inconsistencies across different systems.

Developers frequently encounter issues where an application works correctly on a development machine but fails in another environment due to dependency conflicts or configuration differences.

In addition, modern software development requires faster delivery cycles, automated deployment pipelines, and scalable infrastructure. Without automation and containerization, managing application deployments becomes complex and time-consuming.

Therefore, there is a need for a solution that ensures consistent application deployment, simplified environment configuration, and automated build and deployment workflows.

The **Dockerized Entertainment React Service** project addresses these issues by containerizing the application and integrating DevOps practices such as CI/CD pipelines and cloud deployment.

2. Project Objectives

The main objectives of this project are:

• To develop a web-based entertainment application using React.
• To containerize the application using Docker for portability.
• To automate build and deployment processes using CI/CD pipelines.
• To deploy the application on cloud infrastructure using AWS EC2.
• To demonstrate modern DevOps practices including containerization and automated deployment.

This project also aims to provide practical experience with tools such as Docker, GitHub, Jenkins, and AWS.

3. Scope of the Project

The scope of this project includes the development of a simple entertainment web application and the implementation of DevOps practices for deployment.

The project focuses on:

- Developing a React-based frontend application.
- Containerizing the application using Docker.
- Managing containers using Docker Compose.
- Automating deployment through Jenkins CI/CD pipeline.
- Deploying the application on AWS EC2 cloud infrastructure.

Future improvements may include backend API expansion, database integration, Kubernetes orchestration, and monitoring systems.

Proposed Solution

1. Key Features

The system provides several key features:

- React-based user interface displaying movie information.
- Containerized application deployment using Docker.
- Multi-container configuration using Docker Compose.
- Automated build pipeline using Jenkins.
- Cloud deployment using AWS EC2.
- Scalable architecture suitable for container orchestration.

2. Overall Architecture / Workflow

The overall architecture of the system follows a containerized DevOps workflow.

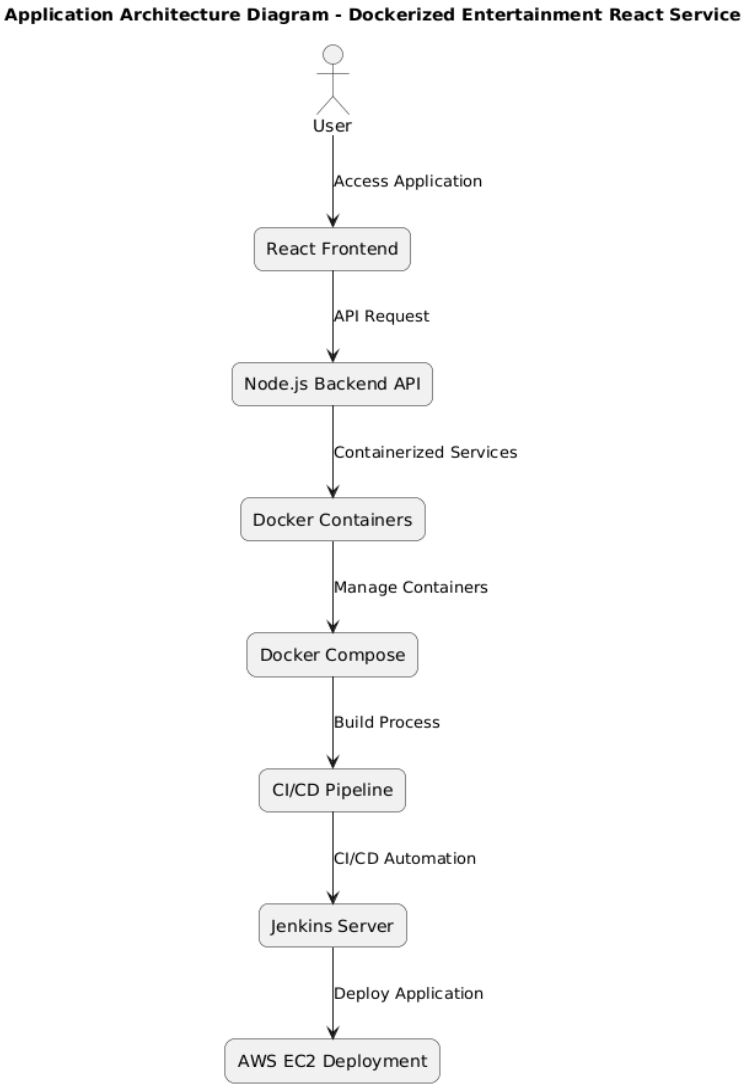


Figure 1: Application Architecture Diagram

This workflow ensures that the application is developed, containerized, and deployed efficiently using DevOps practices.

Application Architecture

The application architecture follows a layered design consisting of three layers:

Client Layer
Container Layer
Application Layer

In the client layer, users interact with the system through a web browser. The browser sends requests to the Docker container hosting the React application.

The container executes the application and returns the user interface to the browser.

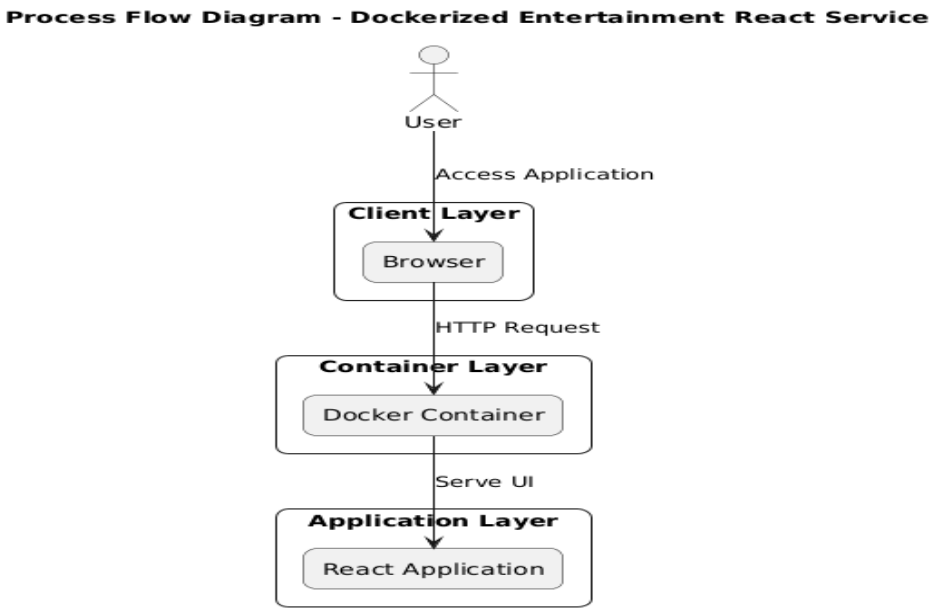


Figure 2: Process Flow Diagram

Information Flow

The information flow describes how data moves between different system components.When the user opens the application URL, the browser sends a request to the Docker container. The container processes the request and retrieves the necessary UI resources from the React application.

The response is then sent back to the browser where the user interface is rendered.

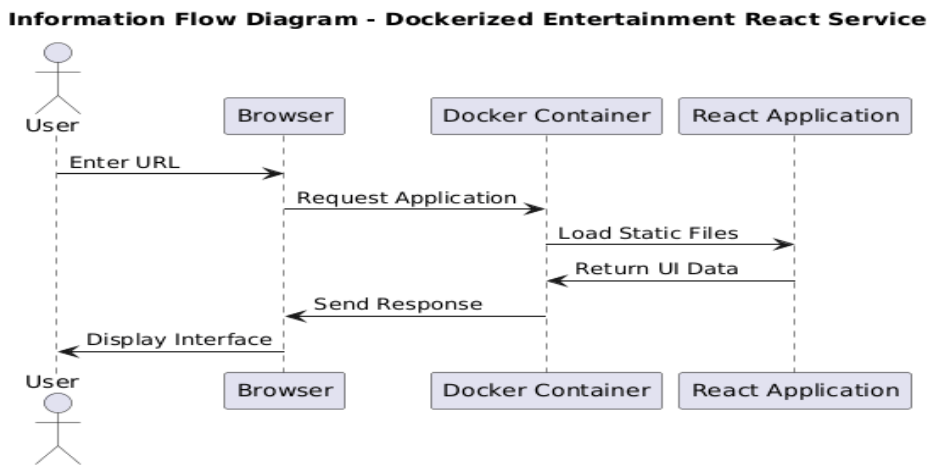


Figure 3: Information Flow Diagram

System Architecture

The system architecture illustrates the interaction between the user, browser, Docker environment, and application components.

The React application runs inside Docker containers, ensuring that the environment remains consistent across development and deployment systems.

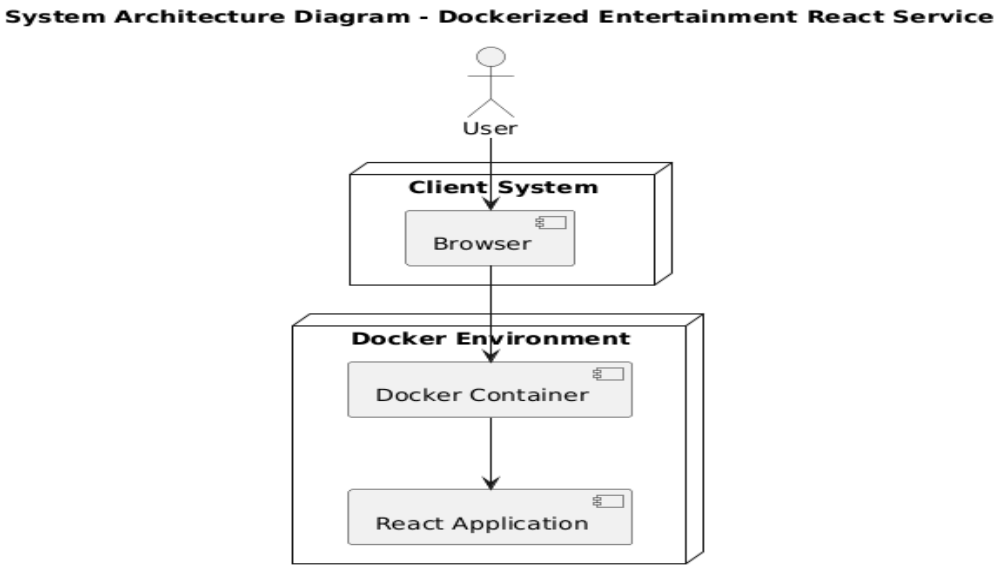


Figure 4: System Architecture Diagram

DevOps Pipeline

The DevOps pipeline automates the build and deployment process.

The workflow begins when developers push code to the GitHub repository. Jenkins detects these changes and triggers a pipeline that builds the Docker image.

Once the image is created, it can be deployed to the cloud infrastructure where the application becomes accessible to users.

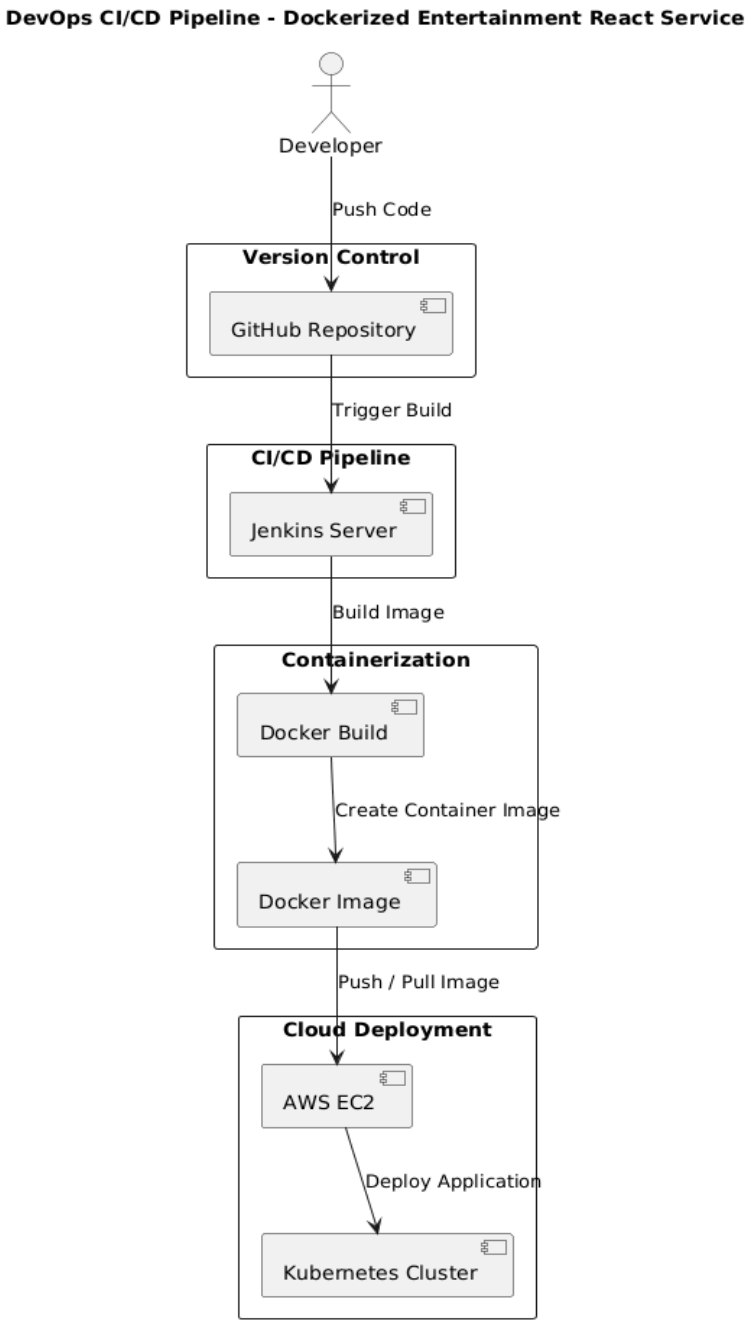


Figure 5: CI/CD Pipeline Diagram

3. Tools & Technologies Used

The project uses several modern technologies:

Frontend Technologies

React.js
Vite
JavaScript

Backend Technologies
Node.js
Express.js

DevOps Tools
Docker
Docker Compose
Jenkins
GitHub

Cloud Infrastructure
AWS EC2

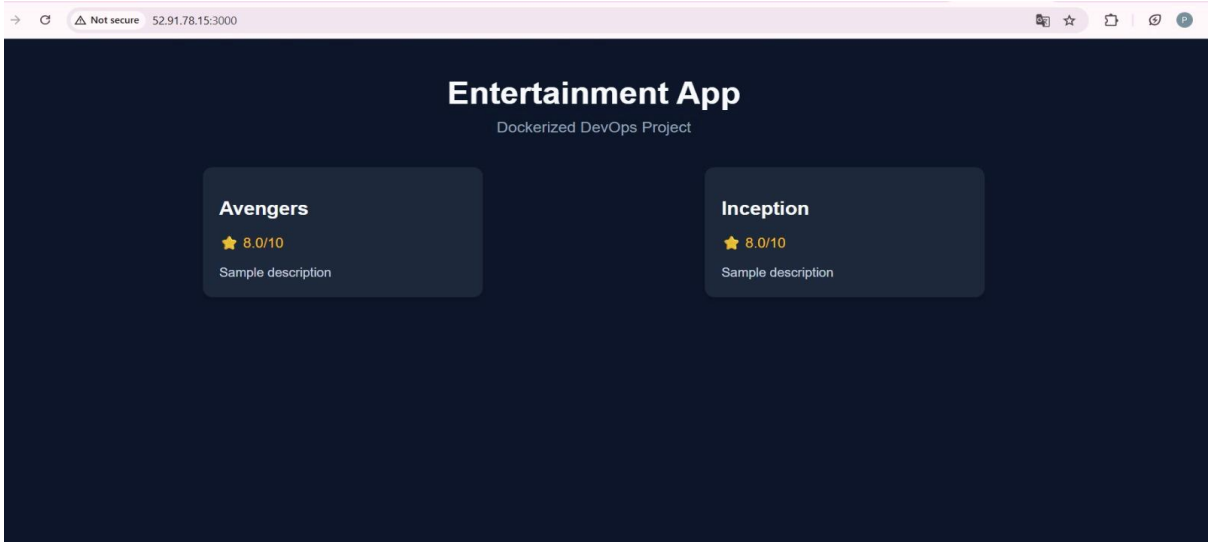
These tools help create a scalable and automated application deployment environment.

Results & Output

1. Screenshots / Outputs

Application UI (Entertainment App Page)

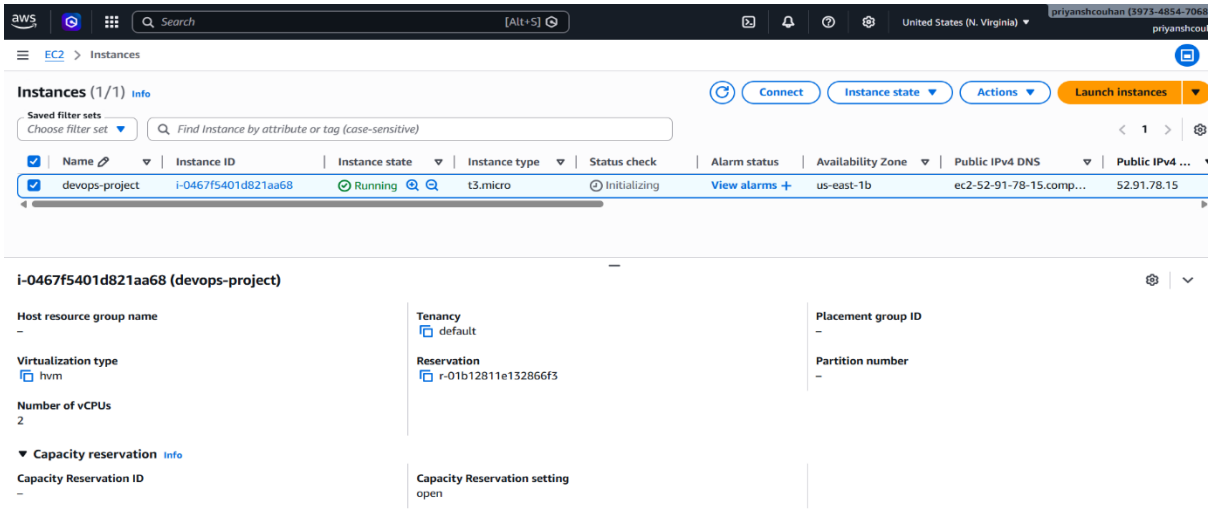
Purpose: Show final working application



Short description: This screenshot shows the user interface of the entertainment application displaying movie titles and ratings.

AWS EC2 Instance Running

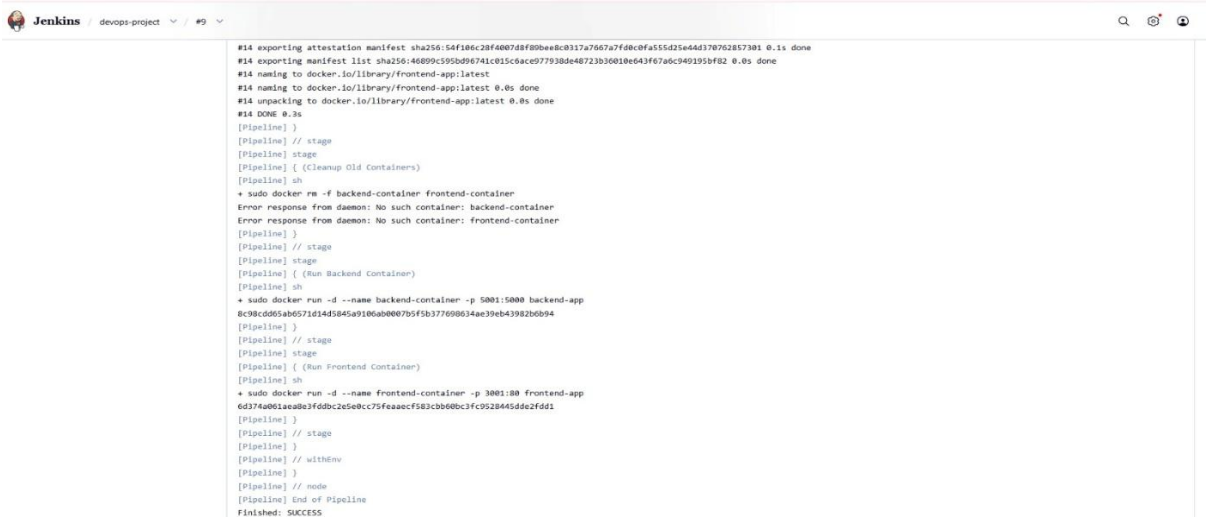
Purpose: Show deployment on cloud



Short description: This screenshot shows the AWS EC2 instance where the application is deployed. The instance is running successfully and provides a public IP address for accessing the application.

Jenkins Pipeline Output

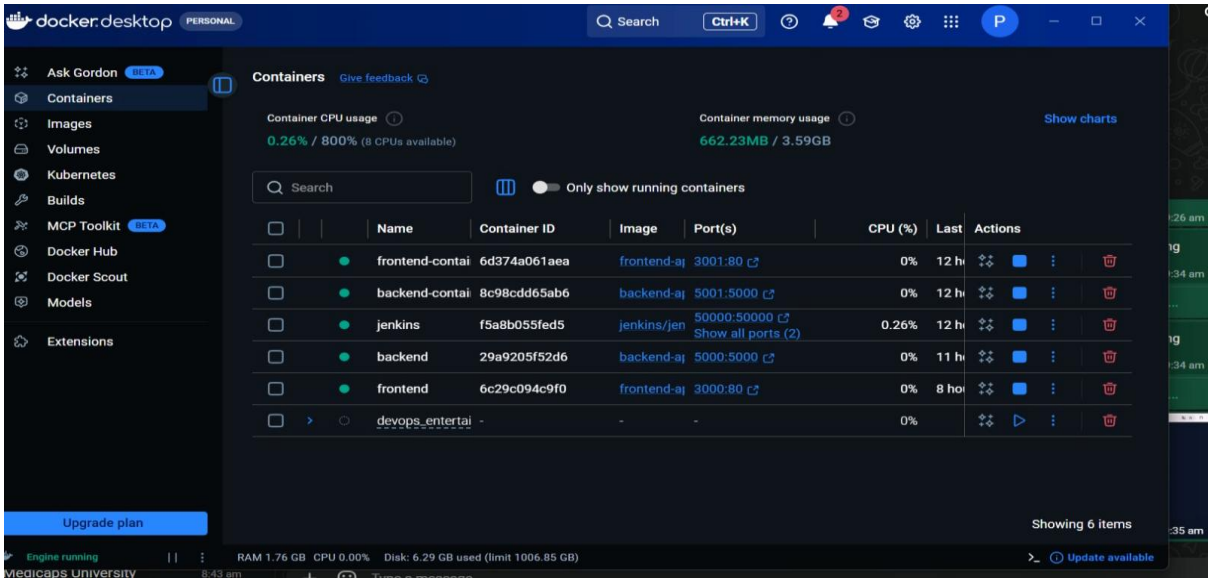
Purpose: Show CI/CD automation



Short description: This screenshot displays the Jenkins pipeline execution where Docker images are built and containers are deployed automatically.

Docker Desktop Containers

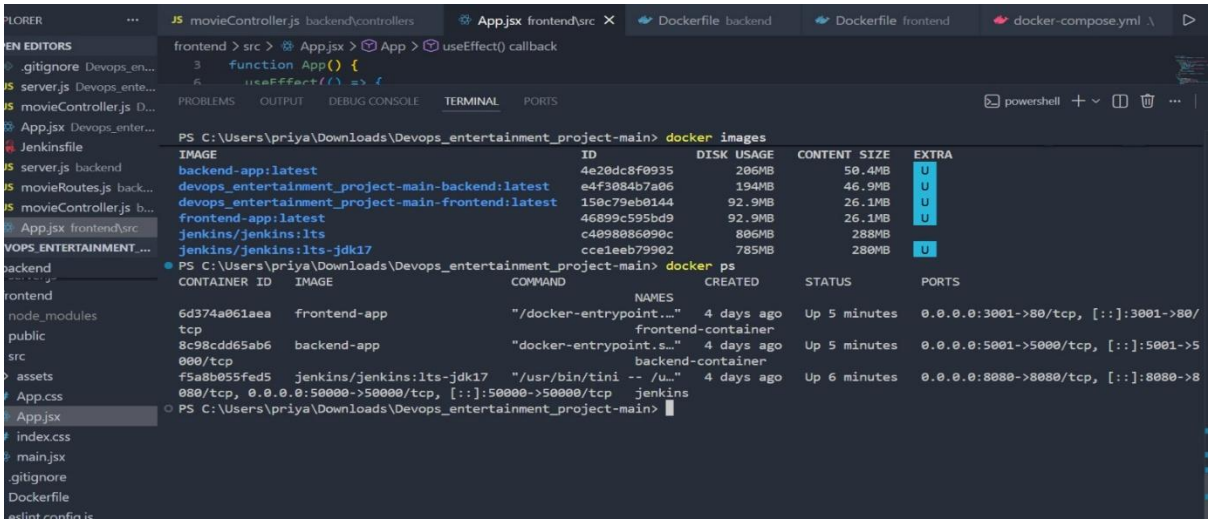
Purpose: Show containerization



Short description: This screenshot shows running Docker containers for the frontend and backend services of the application.

Docker Terminal Output (docker images / docker ps)

Purpose: Show images and running containers



Short description:

This screenshot displays Docker images and running containers created during the build and deployment process.

2. Reports / Dashboards / Models

In this project, reports primarily consist of deployment results and system workflow documentation.
The system architecture diagrams and workflow diagrams provide a clear understanding of how different components interact within the system.

These diagrams illustrate the flow of data between the React frontend, Node.js backend, Docker containers, CI/CD pipeline, and AWS deployment environment.

The diagrams also serve as technical documentation for developers and DevOps engineers who may work with the system in the future, helping them understand the architecture and deployment workflow of the application.

3. Key Outcomes

The project successfully demonstrates the following:

• Development of a React-based entertainment web application.
• Successful containerization of the application using Docker.
• Automated build pipeline using Jenkins.
• Deployment of the application on AWS EC2.
• Implementation of DevOps principles in application development and deployment.

Conclusion

The Dockerized Entertainment React Service project demonstrates the effective use of modern DevOps practices for application development and deployment.

By containerizing the application using Docker, the system ensures portability and consistent execution across different environments. The integration of CI/CD pipelines automates the build process and simplifies deployment workflows.

The deployment of the application on AWS EC2 further demonstrates how cloud infrastructure can be used to host and manage containerized applications.

Overall, this project provides practical experience in implementing DevOps concepts and tools, preparing developers for real-world application deployment scenarios.

Future Scope & Enhancements

Several improvements can be made in future versions of the project.

- Integration of Kubernetes for container orchestration.
- Implementation of monitoring tools such as Prometheus and Grafana.
- Integration of database systems for persistent data storage.
- Implementation of authentication and user management features.
- Infrastructure automation using Terraform.

These enhancements will further improve scalability, automation, and reliability of the system.